3.1.1. Numpy Array Operations

04:38

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Explorer

numpyarr...

Submit

Debugger

```
1  import numpy as np
2
3  # write your code here...
4  rows,cols=map(int,input().split())
```

Average time	Maximum time
0.013 s	0.026 s
12.58 ms	26.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1 26 ms Debug

Expected output	Actual output
3 4	3 4
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
9 10 11 12	9 10 11 12
[[1 2 3 4]	[[1 2 3 4]
[5 6 7 8]	[5 6 7 8]

Terminal Test cases

< Prev Reset Submit Next >

3.2.1. Numpy: Matrix Operations

10:09

The given code takes two 3×3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** ($\text{matrix\_a} + \text{matrix\_b}$)
2. **Subtraction** ($\text{matrix\_a} - \text{matrix\_b}$)
3. **Element-wise Multiplication** ($\text{matrix\_a} * \text{matrix\_b}$)
4. **Matrix Multiplication** ($\text{matrix\_a} \cdot \text{matrix\_b}$)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Explorer

matrixOp...

Submit

```
1  import numpy as np
2
3  # Input matrices
4  print("Enter Matrix A:")
```

Debugger

Average time	Maximum time
0.028 s	0.032 s
28.25 ms	32.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 32 ms Debug

Expected output	Actual output
Enter Matrix A:	Enter Matrix A:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Matrix B:	Enter Matrix B:
1 1 1	1 1 1

Terminal Test cases

< Prev Reset Submit Next >

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

06:02

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking**: Stack the two matrices horizontally (side by side).
- **Vertical Stacking**: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases +

Explorer

stacking.py

Submit

```
1  import numpy as np
2
3  # Input matrices
4  print("Enter Array1:")
```

Debugger

Average time 0.026 s 26.00 ms

Maximum time 0.031 s 31.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 31 ms Debug

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:
4 5 6	4 5 6

Terminal Test cases

< Prev Reset Submit Next >

3.2.3. Numpy: Custom Sequence Generation

03:39

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases +

Explorer

customS...

Submit

```
1  import numpy as np
2
3  # Take user input for the start, stop, and step of the
   sequence
```

Debugger

Average time	Maximum time
0.018 s	0.020 s
18.50 ms	20.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 19 ms Debug

Expected output	Actual output
3	3
10	10
2	2
[3 5 7 9]	[3 5 7 9]

Test case 2 17 ms

Terminal Test cases

< Prev Reset Submit Next >

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematic... 05:17

You are given two arrays `A` and `B`. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. **Arithmetic Operations**:
   - Compute the element-wise sum, difference, and product of the two arrays.
2. **Statistical Operations**:
   - Calculate the mean, median, and standard deviation of array `A`.
3. **Bitwise Operations**:
   - Perform bitwise `AND`, bitwise `OR`, and bitwise `XOR` on the arrays (ex: A_i `OR` B_i).

Input Format:

- The first line contains space-separated integers representing the elements of array `A`.
- The second line contains space-separated integers representing the elements of array `B`.

Output Format:

Explorer

different...

Submit

Debugger

```
1  import numpy as np
2
3  def array_operations(A, B):
4
```

Average time	Maximum time
0.013 s	0.017 s
13.00 ms	17.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 17 ms Debug

Expected output	Actual output
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
Element-wise Sum: 6 8 10 12	Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4	Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32	Element-wise Product: 5 12 21 32
Mean of A: 2.5	Mean of A: 2.5

Terminal Test cases

3.2.5. Numpy: Copying and Viewing Arrays

01:33

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
```

Explorer

copyAnd...

Submit

```
1  import numpy as np
2
3  inputlist = list(map(int,input().split(" ")))
4
```

Debugger

Average time	Maximum time
0.007 s	0.009 s
7.50 ms	9.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 9 ms Debug

Expected output	Actual output
10 20 30 40 50 60 70 80	10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]	Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]	View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]	Original array after modifying copy: [99 20 30 40 50 60 70 80]

Terminal Test cases

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

06:53

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.
Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. **Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
2. **Counting**: Count how many times `count_value` appears in `array1` and print the count.
3. **Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. **Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array

Explorer

arrayOpe...

Submit

Debugger

```
1  import numpy as np
2
3  # Input array from the user
4  array1 = np.array(list(map(int, input().split())))
```

Average time	Maximum time
0.018 s	0.025 s
17.50 ms	25.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 25 ms Debug

Expected output	Actual output
1 1 1 2 2 2	1 1 1 2 2 2
Value to search: 1	Value to search: 1
Value to count: 2	Value to count: 2
Value to add: 2	Value to add: 2
[0 1 2]	[0 1 2]
3	3

Terminal Test cases

< Prev Reset Submit Next >

3.2.7. Student Data Analysis and Operations

18:08

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details**: Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students**: Determine the total number of students in the dataset.
- **Print all student roll numbers**: Extract and print the roll numbers of all students.
- **Print Subject 1 marks**: Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2**: Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3**: Identify the highest marks in Subject 3.
- **Print all subject marks**: Display the marks of all students for each subject.
- **Find total marks of students**: Compute the total marks for each student across all subjects.
- **Find the average marks of each student**: Compute the average marks for each student.
- **Find average marks of each subject**: Compute the average marks for all students in each subject.

Explorer

Operatio...

Submit

Debugger

```
1  import numpy as np
2
3  a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
```

Average time
0.022 s
22.00 ms

Maximum time
0.022 s
22.00 ms

1 out of 1 shown test case(s) passed

Test case 1 22 ms Debug

Expected output

```
All student Details:
[[301.  67.  77.  88.]
 [302.  78.  88.  77.]
 [303.  45.  56.  89.]
 [304.  88.  98.  45.]
 [305.  78.  88.  99.]
```

Actual output

```
All student Details:
[[301.  67.  77.  88.]
 [302.  78.  88.  77.]
 [303.  45.  56.  89.]
 [304.  88.  98.  45.]
 [305.  78.  88.  99.]
```

Terminal Test cases